Final Review & Mock Contest

A year of algorithms in one session — then put it to work

Learning goals

- Review the year's toolbox and how its pieces connect
- Recognize which technique a problem wants from **keywords and constraints**
- Apply the problem-solving method under time pressure
- Sit a **5-hour mock contest** covering the whole year
- Debrief the contest: timeline, root causes, upsolving list
- Plan the next stage (Regional track) and summer training

The year at a glance

Weeks	Block	Highlights
1–9	Foundations	STL, complexity, greedy, binary search, two pointers, prefix sums, backtracking, bits
10–15	Math & DP I	number theory, modular arithmetic, stacks/heaps, DP basics & classics
16–22	Graphs	BFS/DFS, shortest paths, DSU/MST, trees, DAG/SCC, LCA
23–27	Range queries & DP II	segment trees, Fenwick, sparse table, lazy, bitmask/interval/digit DP
28–35	Advanced topics	strings, combinatorics & probability, geometry, flow, games, sqrt, strategy

Foundations (Weeks 1–9)

- **Complexity first:** constraints → target complexity (~10^8 ops/s)
- **Sorting + greedy:** prove with an exchange argument
- **Binary search on the answer:** monotone predicate "can we achieve x?"
- **Two pointers / sliding window:** monotone window boundaries
- **Prefix sums & difference arrays:** O(1) range sums / O(1) range adds offline
- **Backtracking with pruning**, **bitmask enumeration** of subsets

Data structures (Weeks 12–13, 19, 23–25, 34)

Need	Tool
next greater / sliding max	monotonic stack / deque
repeated min/max, k-th smallest online	priority queue, `multiset`, PBDS
merge groups, connectivity offline	DSU
point update + range query	Fenwick / segment tree
static RMQ in O(1)	sparse table
range update + range query	lazy segment tree
weird non-mergeable queries	sqrt decomposition / Mo

Dynamic programming (Weeks 14–15, 20, 26–27)

- Define the **state** (what you must remember) and the **transition** (last decision)
- 1D / 2D classics: knapsack, LIS (O(n log n)), LCS, edit distance, grid paths
- **Tree DP** (children first), rerooting
- **DP on DAGs** = shortest/longest paths in topological order
- **Bitmask DP:** n ≤ 20, state = subset (TSP, assignment)
- **Interval DP:** dp[l][r], O(n^3); **digit DP:** count numbers ≤ N with a property

Graphs (Weeks 16–22, 32)

Question	Algorithm
unweighted shortest path, grids	BFS (0-1 BFS for weights 0/1)
non-negative weights	Dijkstra O((n + m) log n)
negative weights / cycles	Bellman-Ford; all pairs: Floyd-Warshall
connectivity, cycles in undirected	DSU / DFS
cheapest connecting network	Kruskal / Prim
ordering with dependencies	topological sort; SCC for cycles
ancestor / path queries on trees	binary lifting LCA, Euler tour
capacities, disjoint paths, matching	max flow (Dinic), min cut

Math (Weeks 10–11, 30, 33)

- gcd/lcm, sieve, factorization in O(√n)
- Modular arithmetic: fast power, inverses (Fermat), nCr mod p with factorial tables
- Counting: stars and bars, Catalan, **inclusion-exclusion** over subsets
- Probability: **linearity of expectation**, indicator variables; DP on probabilities
- Games: W/L DP, **Nim = XOR**, **Grundy = mex**, sums of games XOR

Strings & geometry (Weeks 28–29, 31)

- Hashing (two moduli / 64-bit with random base), prefix function, Z-function
- Trie, binary trie (max XOR), **Aho-Corasick** for many patterns
- Suffix array + LCP: distinct substrings, longest repeat
- Geometry: everything via **cross product sign**; exact integer predicates
- Segment intersection, shoelace, Pick, ray casting, monotone-chain hull

Keyword → technique (1/2)

You read...	Think...
"minimum maximum", "at least k possible?"	binary search on the answer
"subarray with sum/condition", "longest window"	two pointers, prefix sums + hashing
"number of ways mod 10^9+7"	DP or combinatorics
"n ≤ 20"	bitmask DP / meet in the middle
"queries on ranges with updates"	Fenwick / segment tree (lazy)
"offline range queries, add/remove easy"	Mo's algorithm
"k-th / next greater / nearest smaller"	monotonic stack, ordered set

Keyword → technique (2/2)

You read...	Think...
"grid, minimum steps"	BFS (multi-source, 0-1 BFS)
"tree, path between u and v"	LCA, prefix sums on root paths
"connected after adding edges"	DSU
"assign, each at most once, maximize"	bipartite matching / flow
"remove minimum edges to separate"	min cut
"players alternate, optimal play"	game DP, Nim, Grundy
"expected value"	linearity of expectation, expected-value DP

The problem-solving method (Week 18 revisited)

1. **Understand**: restate the problem in one sentence; what exactly is asked?
2. **Examples**: work the samples and a few tiny cases by hand
3. **Brute force**: what is the obvious slow solution? (Keep it — it is your stress tester!)
4. **Observe**: monotonicity? independence? small parameter? invariant?
5. **Optimize**: pick the data structure / technique that removes the bottleneck
6. **Verify**: complexity check, edge cases, stress test

The year's top bugs

1. `int` overflow (sums, products, `1 << 40`)
2. Multi-test state not reset
3. Off-by-one in ranges and binary search bounds
4. Recursion too deep (DFS on 10^6 vertices)
5. Wrong INF (1e9 when distances reach 10^{14})
6. Dijkstra without skipping stale heap entries
7. Forgetting the reverse edge (flow) / both directions (undirected graphs)
8. Negative results from `%`
9. Greedy without proof
10. Output format: case, spacing, precision

Mock contest: rules

- **5 hours**, ICPC rules: solved count, then penalty (20 min per wrong try)
- Teams of 3, **one computer** — or individually for extra practice
- Problems are **not** sorted by difficulty: triage first!
- Allowed: your notebook / printed reference, C++ docs
- After the contest: 1-hour debrief, then upsolve everything

All problems are on CSES (https://cses.fi/problemset), so you can submit and get verdicts.

Mock contest: problem set

	Problem	CSES
A	Concert Tickets	1091
B	Hamiltonian Flights	1690
C	Sum of Two Values	1640
D	Company Queries II	1688
E	Prime Multiples	2185
F	Dice Combinations	1633
G	Download Speed	1694
H	String Matching	1753
I	Road Reparation	1675

Debrief

> **Solutions are not published here.** The coach holds the solutions and releases them only after the contest has been run.

- Rebuild the team timeline: read, started, submitted, accepted — per problem
- Root cause of every rejection: misread, wrong idea, bug, overflow, TLE, format
- Compare your solve order with the rest of the room: which problem did you find late?
- Write the upsolving list before anyone leaves

Self-assessment

After upsolving, rate yourself 1–5 on each block:

Block	Can I implement it from memory in 15 min?
BFS/Dijkstra/DSU	
segment tree (lazy) / Fenwick	
DP classics, bitmask DP	
LCA / Euler tour	
KMP / Z / hashing / Aho-Corasick	
Dinic / Kuhn	
geometry primitives + hull	

Anything below 4 → re-solve that week's practice list.

What comes next

- **Regional track** (02-Regionals): implementation-heavy problems, advanced greedy, graph modeling, DP optimizations, sweep line, number theory II, geometry & strings for regionals
- Later: **National** and **World Finals** tracks (persistent structures, HLD/centroid, flows II, suffix automaton, FFT, ...)
- The biggest jump comes from **volume + upsolving**, not from new topics

Summer training plan

Every week	Amount
virtual contest (Codeforces Div. 2 / AtCoder ABC–ARC / ICPC gym set)	1–2
upsolve problems you almost solved	all of them
topic practice from this year's lists	5–10 problems
notebook: add/test one template	1

Track your problems and mistakes in a log; review it before each contest.

Summary

- One year: foundations → data structures → DP → graphs → math → strings/geometry → flows/games/sqrt
- Recognize techniques from **constraints and keywords**
- Method: understand → examples → brute force → observe → optimize → verify
- Mock contest: triage, team play, stress testing — then **upsolve everything**

Congratulations on finishing the Year Curriculum!